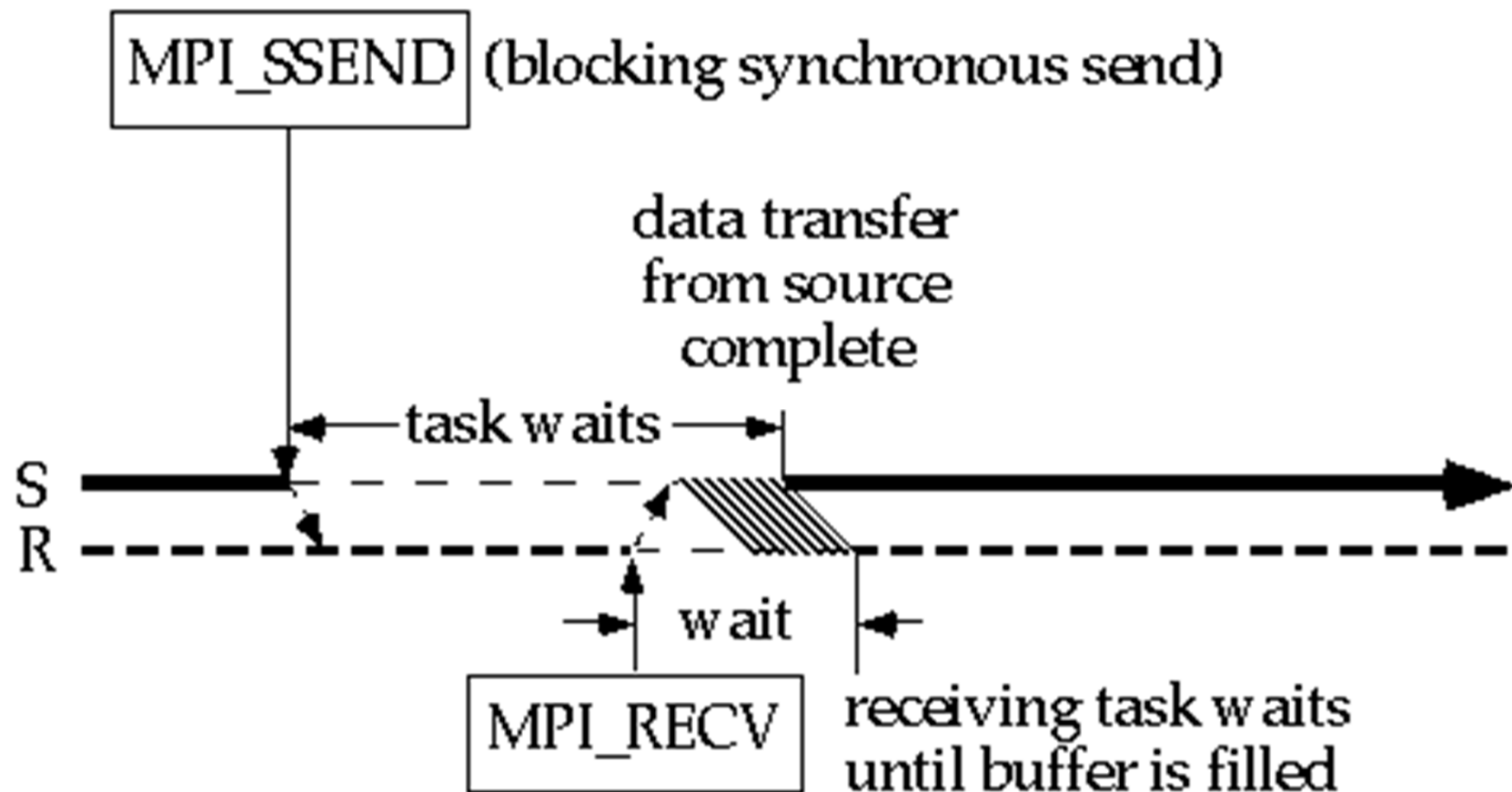




Способы коммуникации

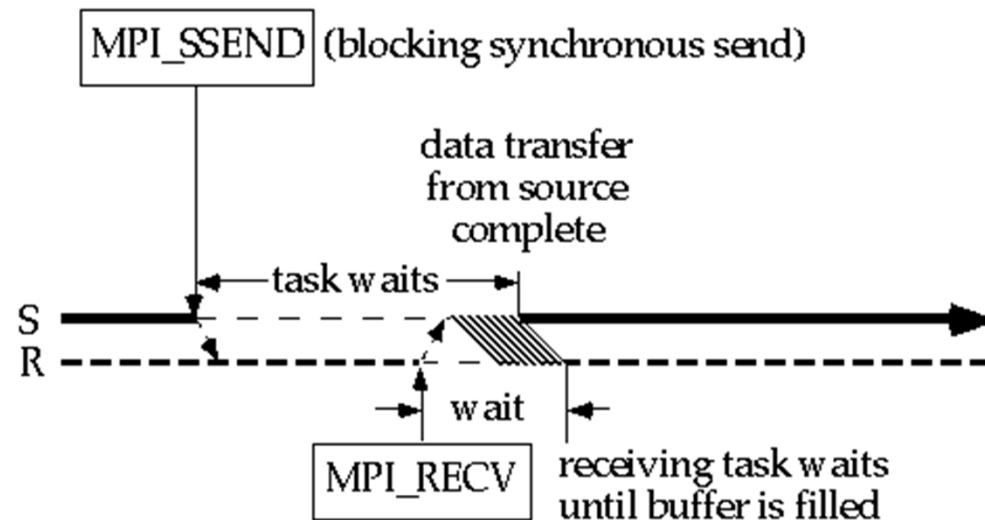
- *Способ коммуникации* – метод, по которому система обрабатывает сообщения
- Способ коммуникации определяется при отправке сообщения
- Существуют 4 способа коммуникации
 - *стандартный (standard)* – MPI_Send
 - *синхронный (synchronous)* – MPI_Ssend
 - *буферизованный (buffered)* – MPI_Bsend
 - *по готовности (ready)* – MPI_Rsend
- Функция получения не определяет способ коммуникации (всегда MPI_Recv)

Синхронная отправка сообщения

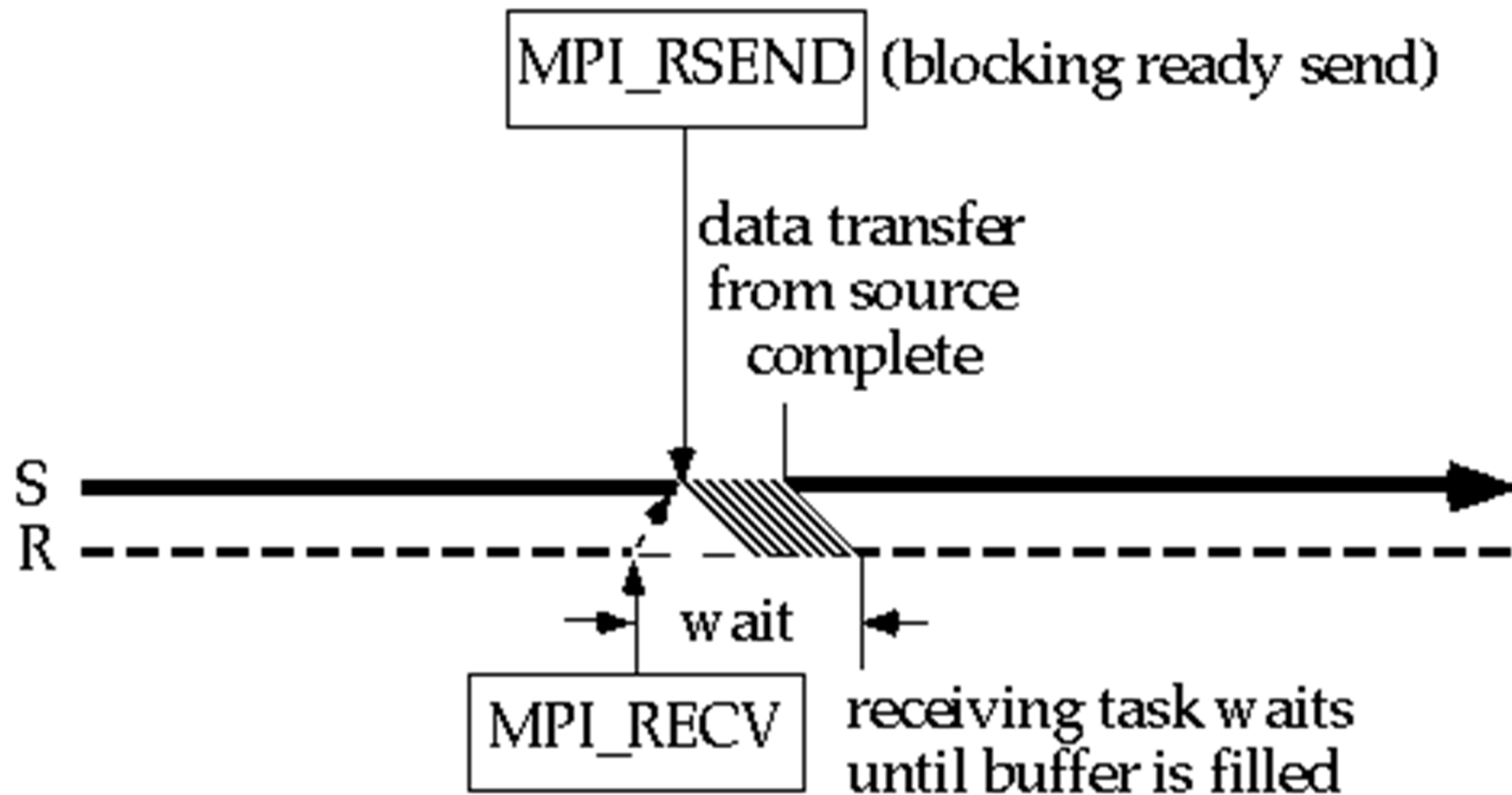


Синхронная отправка сообщения

- Существуют два источника задержки при передаче сообщения
 - **системная накладка** – вызывается копированием данных сообщения из буфера сообщения отправителя и копированием данных из сети в буфер сообщения получателя
 - **синхронизационная накладка** – время, потраченное на ожидание другой задачи

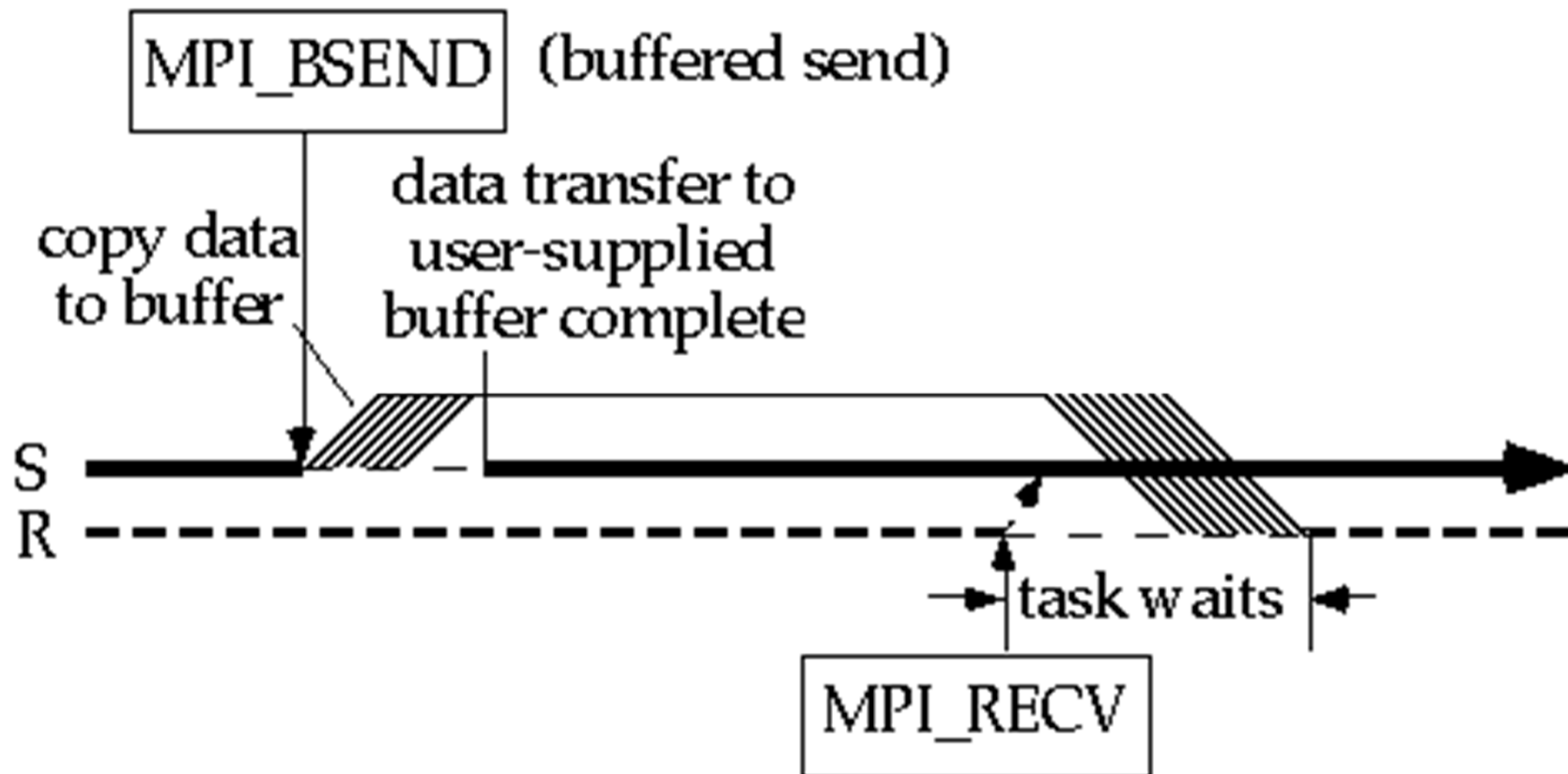


Отправка сообщения по готовности



- Если операция приёма не начала выполнение, то поведение не определено (выдаст ошибку)

Буферизованная отправка сообщения



- На стороне отправителя выделяется дополнительный буфер, в котором отправляемые данные хранятся до момента их получения



Буферизованная отправка сообщения

- Для использования буферизованного режима передачи должен быть создан и передан MPI буфер памяти для буферизации сообщений
 - `int MPI_Buffer_attach( void* buffer, int size )`
 - `buffer` – буфер памяти для буферизации сообщений
 - `size` – размер буфера в байтах
- После завершения работы с буфером он должен быть отключен от MPI при помощи функции
 - `int MPI_Buffer_detach( void* buffer, int size )`



Буферизованная отправка сообщения

```
int BUFSIZE = sizeof(int) + MPI_BSEND_OVERHEAD;  
char *buf = new char[BUFSIZE] {0};
```

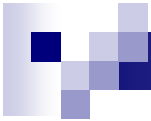
```
int buf_size; //размер буфера  
int proc_rank; //ранг процесса
```

```
int num; //число
```

```
MPI_Status st;
```

```
MPI_Init(&argc, &argv); //начало области MPI
```

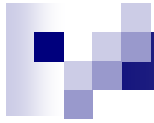
```
MPI_Comm_rank(MPI_COMM_WORLD, &proc_rank); //ранг
```



Буферизованная отправка сообщения

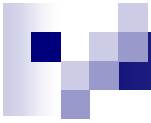
```
if (proc_rank == 0) { //0 процесс
    cin >> num;
    MPI_Buffer_attach(buf, BUFSIZE); //присоединяем буфер
    MPI_Bsend(&num, 1, MPI_INT, 1, 100, MPI_COMM_WORLD);
    MPI_Buffer_detach(buf, &buf_size); //отсоединяем
}
if (proc_rank == 1) { //1 процесс
    MPI_Recv(&num, 1, MPI_INT, 0, 100, MPI_COMM_WORLD,
    &st);
    cout << "Process 1 received number " << num << " from
    process " << st.MPI_SOURCE << endl;
}

MPI_Finalize();
```

Стандартная отправка сообщения

- Решение о том, будет ли исходящее сообщение буферизовано, принимает MPI
- Посылка может зависеть от условий приёма и размера передаваемого сообщения



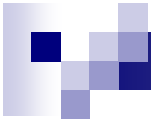
Неблокирующие обмены

- Часть сообщений может быть отправлена и принята заранее до момента реальной потребности в пересылаемых данных
- Коммуникационные операции разделяются на две стадии
 - инициирование операции
 - проверка завершения операции
- Использование неблокирующих операций повышает безопасность программы с точки зрения возникновения тупиковых ситуаций, а также может увеличить скорость работы программы за счёт совмещения выполнения вычислительных и коммуникационных операций
- Замечание. Многое зависит от реализации, не всегда асинхронные операции поддерживаются аппаратурой и системным окружением



Неблокирующие обмены

- Используем префикс I (*Immediate*)
- ```
int MPI_Isend(void* buf, int count, MPI_Datatype type,
 int dest, int tag, MPI_Comm
 MPI_Request* request)
```
- Дополнительный параметр request типа MPI\_Request используется для идентификации конкретной неблокирующей операции
- Возврат из функции происходит сразу после инициализации процесса передачи без ожидания обработки всего сообщения, находящегося в буфере
- Определить момент времени, когда можно повторно использовать буфер, можно при помощи функций MPI\_Wait и MPI\_Test

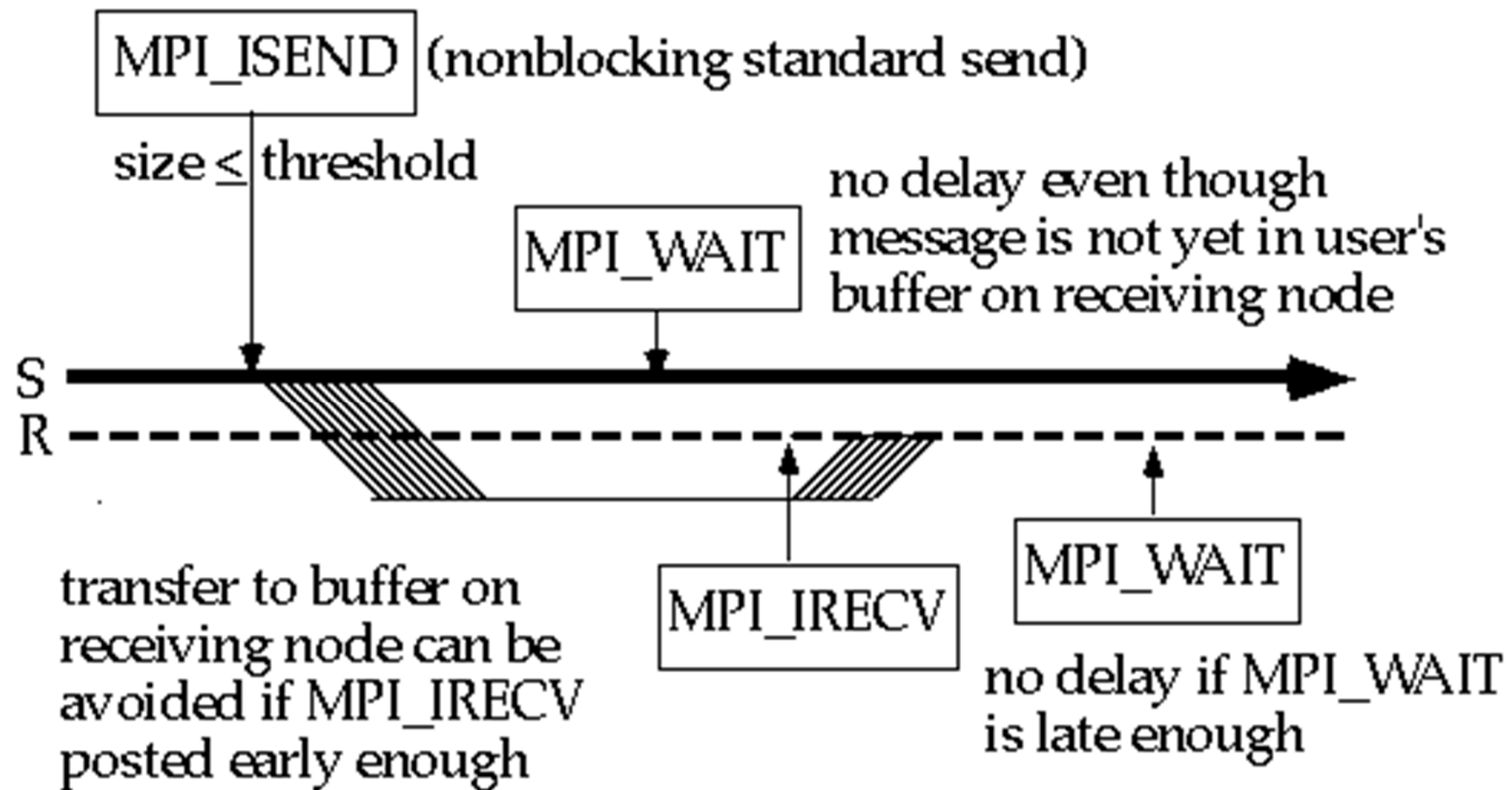


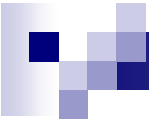
# Неблокирующие обмены

- `MPI_Irecv( void* buf, int count, MPI_Datatype type,  
int dest, int tag, MPI_Comm  
MPI_Request* request )`
- Возврат из процедуры происходит сразу после инициализации процесса приёма без ожидания получения всего сообщения и его записи в буфер
- Окончание приёма также можно определить при помощи функций `MPI_Wait` и `MPI_Test`
- Операция неблокирующей отправки имеет три дополнительных варианта
  - `MPI_Isend`
  - `MPI_Ibsend`
  - `MPI_Irsend`

# Неблокирующая стандартная отправка

- MPI\_Isend / MPI\_Irecv





# Неблокирующие обмены

- `int MPI_Wait(MPI_Request *request, MPI_Status *status)`
  - блокирует процесс до тех пор, пока асинхронная операция с параметром `request` не будет завершена
  - `request` – дескриптор операции
  - `status` – результат выполнения операции обмена
- После выполнения идентификатор неблокирующей операции `request` устанавливается в значение `MPI_REQUEST_NULL`
- Другие варианты операции ожидания:
  - `int MPI_Waitany(int count, MPI_Request requests[], int *index, MPI_Status *status)`
  - `int MPI_Waitall(int count, MPI_Request requests[], MPI_Status statuses[])`
  - `int MPI_Waitsome(int incount, MPI_Request requests[], int *outcount, int indices[], MPI_Status statuses[])`



# Неблокирующие обмены

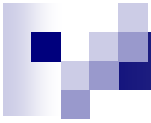
- `int MPI_Test(MPI_Request *request, int *flag, MPI_Status *status)`
  - `flag` – результат проверки (`true/false`)
- Проверяет завершенность асинхронной операции, ассоциированной с идентификатором `request`
  - `true` – завершена
  - `false` – не завершена
- Если операция завершена, то будет проинициализирована переменная `status`
- После выполнения идентификатор неблокирующей операции `request` устанавливается в значение `MPI_REQUEST_NULL`
- Другие варианты операции:  
`MPI_Testany`, `MPI_Testall`, `MPI_Testsome`



# Неблокирующие обмены

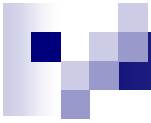
- `int MPI_Probe(int source, int tag, MPI_Comm comm, MPI_Status *status)`
- `MPI_Iprobe`
- `int MPI_Cancel(MPI_Request *request)`
- `int MPI_Test_cancelled(const MPI_Status *status, int *flag)`





# Отложенные запросы

- Позволяют снизить накладные расходы, возникающие при обработке приёма/передачи на одном процессоре и перемещении необходимой информации между процессором и сетевым контроллером
- Если нужно многократно выполнять обмены с одинаковыми параметрами, то можно один раз инициализировать операции обмена, а потом многократно их запускать
- Таким образом можно объединять несколько запросов на приём и/или передачу
- ```
int MPI_Send_init(void* buf, int cnt, MPI_Datatype type,
                  int dest, int tag, MPI_Comm comm,
                  MPI_Request* request);
```
- ```
int MPI_Recv_init(...)
```



# Отложенные запросы

- Инициализация отложенного запроса на выполнение операции обмена, соответствующей значению параметра request
- `int MPI_Start( MPI_Request* request )`
- `int MPI_Startall( int count, MPI_Request* array_of_requests )`
- Удаление структур данных, связанных с параметром request
- `int MPI_Request_free(MPI_Request* request)`
- После её выполнения параметр request устанавливается в значение `MPI_REQUEST_NULL`
- Если выполняется операция, связанная с этим запросом, то она будет завершена